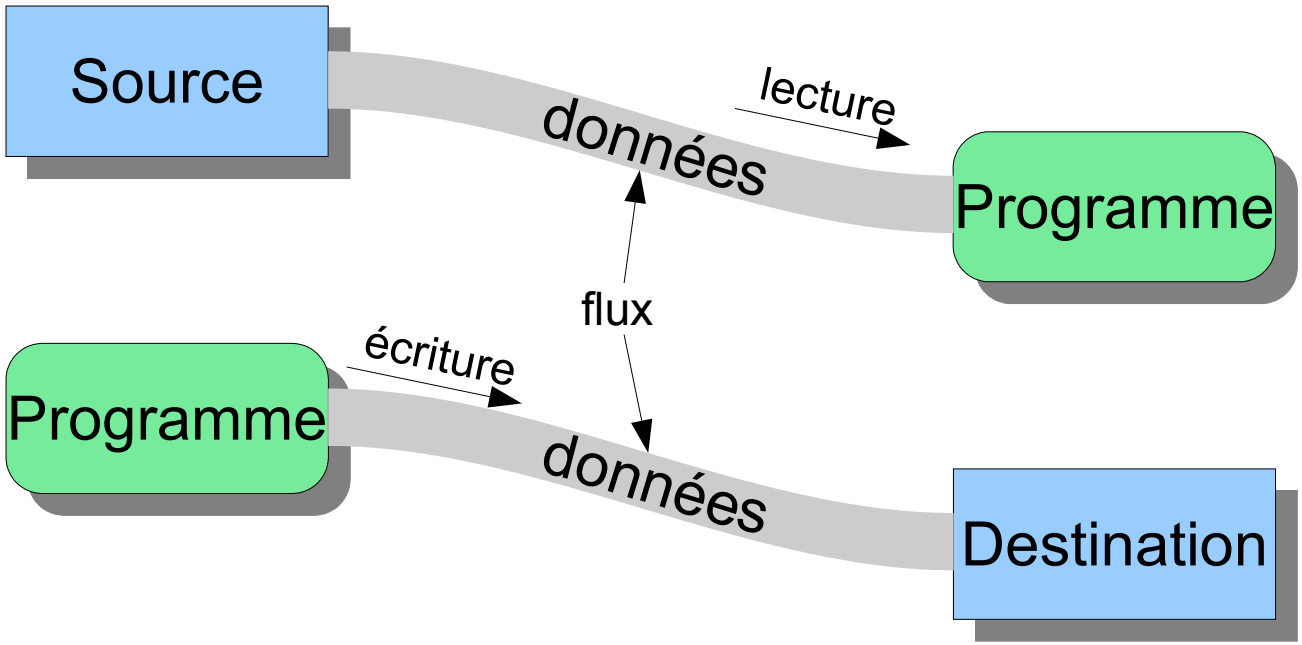
# Programmation
# - Gestion des entrées / sorties -

Nicolas Malandain

1. Présentation
   - Flux
   - Types de flux
   - Homogénéité des classes de base
2. Flux octet
   - Présentation
   - Description
   - Exemples
3. Flux caractère
   - Présentation
   - Description
   - Exemples
4. Sérialisation
   - Présentation
   - Écriture/Lecture
   - Contrôle

# Principe des flux




## Remarques

- Source/Destination : “programme”, fichier (texte, binaire), BD, ...
- indépendance vis à vis du type des données
- un flux est unidirectionnel
- les méthodes d’accès aux flux sont identiques

# Deux types de flux

API des flux dans le package `java.io`

## Flux orientés octet (binaire)

- manipulation d'informations binaires sur 8 bits
- à utiliser pour lire/écrire des images, sons, ...

## Flux orientés caractère (texte)

- manipulation d'informations de type caractère sur 16bits (unicode)
- à utiliser pour lire/écrire du texte

## Remarque

Pour l'accès à des fichiers sur disque, les flots bufferisés sont fortement conseillés (cf. transparents suivants).

# Flux de lecture

## `InputStream` classe mère de l'API orienté octet

- `int read()`
- `int read(byte cbuf[])`
- `int read(byte cbuf[], int offset, int length)`

## `Reader` classe mère de l'API orienté caractère

- `int read()`
- `int read(char cbuf[])`
- `int read(char cbuf[], int offset, int length)`

# Flux d'écriture

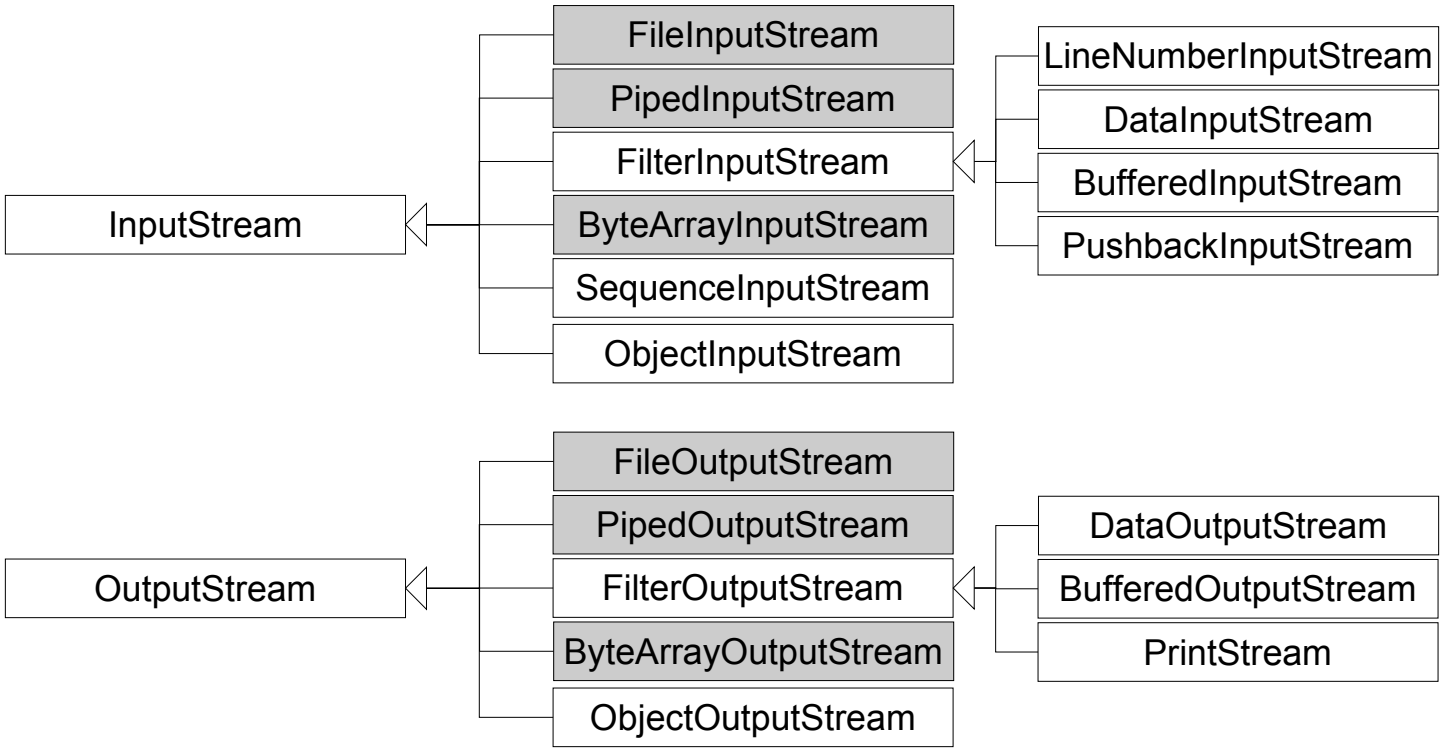
## `OutputStream` classe mère de l'API orienté octet

- `int write(int c)`
- `int write(byte cbuf[])`
- `int write(byte cbuf[], int offset, int length)`

## `Writer` classe mère de l'API orienté caractère

- `int write(int c)`
- `int write(char cbuf[])`
- `int write(char cbuf[], int offset, int length)`

# Hiérarchie des classes




*En gris les classes liés à des conteneurs de données*
*En blanc les classes liés à des traitements des données*

# Classes en lecture

## Conteneurs de données

| | |
|---|---|
| FileInputStream | lecture de fichiers (octet par octet) |
| PipedInputStream | lecture des données venant d'un `PipedOutputStream` |
| ByteArrayInputStream | lecture de données dans un tampon (tableau) en mémoire |

## Traitement de données

| | |
|---|---|
| SequenceInputStream | concaténation de flux en un seul |
| ObjectInputStream | lecture d'objets (désérialisation d'objets java) |
| FilterInputStream | lecture en modifiant certaines données (numérotation, conversions de données, bufferisation, vue en avant, . . . ) |

# Classes en écriture

## Conteneurs de données

| | |
|---|---|
| FileOutputStream | écriture de fichiers (octet par octet) |
| PipedOutputStream | écriture dans un flux lu par un `PipedInputStream` |
| ByteArrayOutputStream | écriture de données dans un tampon (tableau) en mémoire |

## Traitement de données

| | |
|---|---|
| ObjectOutputStream | écriture d'objets (sérialisation d'objets java) |
| FilterOutputStream | écriture en modifiant certaines données (conversions de données, bufferisation, facilité d'affichage/formatage, ...) |

# Les flux standards

## Les 3 flux standards

Attributs de classe de `System` (`java.util`)

- `System.in` entrée standard (`InputStream`)
- `System.out` sortie standard (`PrintStream`)
- `System.err` sortie d'erreur standard (`PrintStream`)

## `System.out` / `PrintStream`

`PrintStream` hérite de `FilterOutputStream`

Fonctionnalités supplémentaires de `PrintStream` : surcharges de `print` et `println` pour écrire des `String`, `byte`, `int`, `char`, `float`, ...

## `System.in` / `InputStream`

`InputStream` lit uniquement des octets. Nécessité d'une couche supérieure pour "décoder" les octets lus, depuis Java 5 $\Rightarrow$ `Scanner`

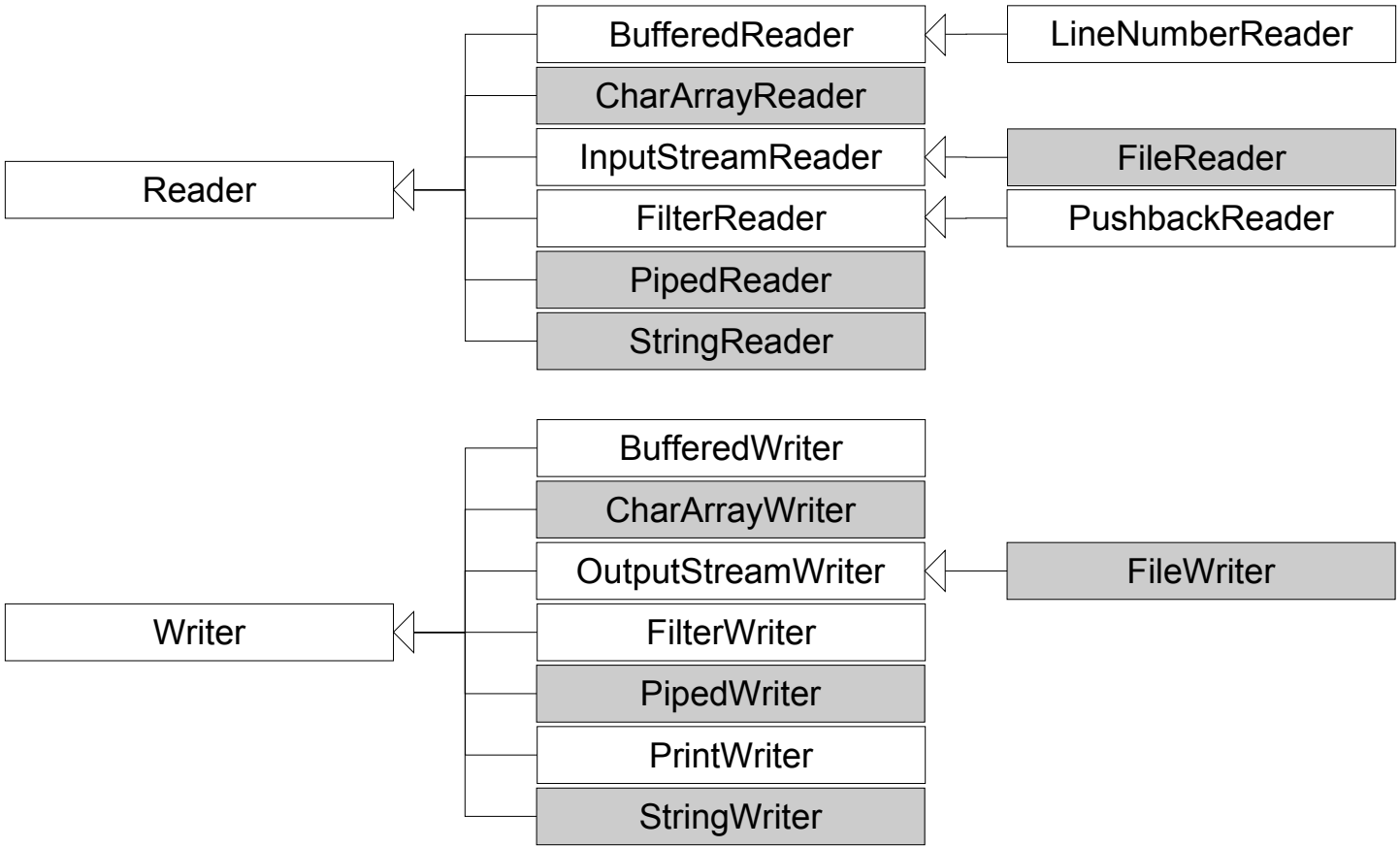
## TestFluxOctet.java

```
Scanner sc1 = new Scanner(System.in);
int nbentier;
double nbdouble;
DataOutputStream dos = new DataOutputStream(new FileOutputStream("fichierbinaire"));
do {
  System.out.print("Entier␣?␣");
  nbentier = sc1.nextInt();
  System.out.print("Double␣?␣");
  nbdouble = sc1.nextDouble();
  dos.writeInt(nbentier);
  dos.writeDouble(nbdouble);
  System.out.println("Continuer␣?␣O/N");
} while (sc1.next("[oOnN]").toUpperCase().equals("O"));
dos.close();

DataInputStream dis = new DataInputStream(new BufferedInputStream(
                          new FileInputStream(new File("fichierbinaire"))));
while (dis.available()!=0) {
  System.out.println("entier␣lu␣:␣"+dis.readInt());
  System.out.println("double␣lu␣:␣"+dis.readDouble());
}
dis.close();

Scanner sc2 = new Scanner(args[0]).useDelimiter("[\\s\\-]+"); // args[0]="73 37,0"
int nbentier2 = sc2.nextInt();
double nbdouble2 = sc2.nextDouble();
System.out.printf("entier␣:␣%d,double␣:␣%.2f␣",nbentier2,nbdouble2);
```

# Hiérarchie des classes




*En gris les classes liés à des conteneurs de données*
*En blanc les classes liés à des traitements des données*

# Classes en lecture

## Conteneurs de données

| | |
|---|---|
| CharArrayReader | lecture de caractères dans un tampon (tableau) en mémoire |
| FileReader | lecture de caractères dans un fichier texte |
| PipedReader | lecture de caractères venant d'un `PipedWriter` |
| StringReader | lecture de caractères à partir d'une `String` |

## Traitement de données

| | |
|---|---|
| BufferedReader | bufferisation de la lecture d'un flux caractère |
| InputStreamReader | conversion de flux octet en flux caractère |
| FilterReader | lecture en modifiant certain caractères (vue en avant, . . . ) |

# Classes en écriture

## Conteneurs de données

| | |
|---|---|
| CharArrayWriter | écriture de caractères dans un tampon (tableau) en mémoire |
| FileWriter | écriture de caractères dans un fichier texte |
| PipedWriter | écriture de caractères dans un flux lu par un `PipedReader` |
| StringWriter | écriture de caracères dans une `String` |

## Traitement de données

| | |
|---|---|
| BufferedWriter | bufferisation de l'écriture dans un flux caractère |
| OutputStreamWriter | conversion de flux caractère en flux octet |
| FilterWriter | écriture en modifiant certains caractères |
| PrintWriter | écriture avec facilité de formatage |

## TestFluxCaractere.java

```
public static void main(String[] args) throws IOException {
  Reader reader = new BufferedReader(new FileReader(args[0]));
  Writer writer = new BufferedWriter(new FileWriter(args[1]));
  PrintWriter pwriter = new PrintWriter(System.out);

  int c;
  while ( (c = reader.read()) != -1) {
    pwriter.write(c);
    writer.write(c);
  }
  pwriter.flush();
  reader.close();
  writer.close();
}
```

# Principe de la sérialisation

## Sérialisation

Permet l'envoi d'objets à travers des flux, notamment des flux de fichiers permettant ainsi la persistance.

## En java

La sérialisation d'objets est notamment apparu dans Java en support à la technique Remote Method Invocation.

- tout objet implémentant l'interface `Serializable` est sérialisable en une suite d'octets, qui pourra être relue pour reconstruire l'objet.
- Java prend en charge la sérialisation des réseaux d'objets (tableaux d'objets, objets d'objets, ...)
- la sérialisation est indépendante de la plateforme
- ni les attributs `transient`, ni les `static` ne sont sérialisés
- à la désérialisation les constructeurs ne sont pas utilisés

# Flots de sérialisation

## ObjectOutputStream / ObjectInputStream

Ces deux flots permettent l'écriture d'objets dans des flots, mais aussi les types simples

- `writeObject` / `readObject`
- `writeInt` / `readInt`
- ...

## Faire qu'un objet soit sérialisable

La classe doit implémenter l'interface `Serializable` qui ne contient aucune méthode.

# Exemple 1

## ExempleSerialisationObjetsEcriture.java

```
FileOutputStream fichier = new FileOutputStream(args[0]);
ObjectOutputStream flotObjets = new ObjectOutputStream(fichier);
flotObjets.writeObject("La réponse à la question ultime de la vie, de l'univers
                        et de toutes choses est ");
flotObjets.writeInt(42);
flotObjets.flush();
```

## ExempleSerialisationObjetsLecture.java

```
FileInputStream fichier = new FileInputStream(args[0]);
ObjectInputStream flotObjets = new ObjectInputStream(fichier);
try {
  String question = (String)flotObjets.readObject();
  int reponse = flotObjets.readInt();
  System.out.println(question+reponse);
} catch (ClassNotFoundException e) {
  System.err.println("Impossible de trouver la classe de l'objet
                      que vous tentez de désérialiser");
  e.printStackTrace();
}
```

# Exemple 2

## ObjetTest.java

```
public class ObjetTest implements Serializable {
  private StringBuilder chaine;
  private int nombre;
  public ObjetTest(StringBuilder chaine, int nombre) {
    this.chaine = chaine;
    this.nombre = nombre;
  }
  public String toString() {
    return "["+super.toString()+"]("+chaine+"---"+nombre+")";
  }
  public StringBuilder getChaine() {
    return chaine;
  }
}
```

## TestsSerialisation.java (...)

```
    StringBuilder chaine = new StringBuilder("La chaîne Lassie");
    ObjetTest[] objTest, objTest1, objTest2, objTest3;
    objTest = new ObjetTest[2];
    objTest[0] = new ObjetTest(chaine,1);
    objTest[1] = new ObjetTest(chaine,2);
    System.out.println("objTest -> "+objTest + Arrays.toString(objTest));
```

## TestsSerialisation.java (...)

```
// sérialisation
ObjectOutputStream oos1 = new ObjectOutputStream(new FileOutputStream("/tmp/fichier1"));
oos1.writeObject(objTest);
oos1.writeObject(objTest);
oos1.close();
ObjectOutputStream oos2 = new ObjectOutputStream(new FileOutputStream("/tmp/fichier2"));
oos2.writeObject(objTest);
oos2.close();

// désérialisation
ObjectInputStream ois1 = new ObjectInputStream(new FileInputStream("/tmp/fichier1"));
objTest1 = (ObjetTest[])ois1.readObject();
objTest1[0].getChaine().replace(3,9,"chienne");
objTest2 = (ObjetTest[])ois1.readObject();
System.out.println("objTest 1 -> "+objTest1 + Arrays.toString(objTest1));
System.out.println("objTest 2 -> "+objTest2 + Arrays.toString(objTest2));
ois1.close();

ObjectInputStream ois2 = new ObjectInputStream(new FileInputStream("/tmp/fichier2"));
objTest3 = (ObjetTest[])ois2.readObject();
System.out.println("objTest 3 -> "+objTest3 + Arrays.toString(objTest3));
ois2.close();
```

## Exécution

```
objTest -> ObjetTest;@7d772e[[ObjetTest@11b86e7](La chaîne Lassie---1),
                             [ObjetTest@35ce36](La chaîne Lassie---2)]

objTest1 -> ObjetTest;@89ae9e[[ObjetTest@1270b73](La chienne Lassie---1),
                              [ObjetTest@60aeb0](La chienne Lassie---2)]

objTest2 -> ObjetTest;@89ae9e[[ObjetTest@1270b73](La chienne Lassie---1),
                              [ObjetTest@60aeb0](La chienne Lassie---2)]

// le stockage d'un reseau d'objets doit être fait dans un même flux
// sinon il y a duplications d'objets
objTest3 -> ObjetTest;@16caf43[[ObjetTest@66848c](La chaîne Lassie---1),
                               [ObjetTest@8813f2](La chaîne Lassie---2)]
```

# Non sérialisation d'attributs

## `transient`

ce mot clé associé à un attribut spécifie que ce dernier ne doit pas être sérialisé

## Utilisation

Peut être utilisé notamment lorsque la sécurité est importante, exemple : ne pas sérialiser un mot de passe $\Rightarrow$ `transient`

# Personnaliser la sérialisation

## Objectif

Modifier le comportement de base de la sérialisation

## Technique

Implémenter `Serializable` et **ajouter** les méthodes suivantes :

```
private void writeObject(ObjectOutputStream s) throws IOException {
  s.defaultWriteObject(); // pour le comportement par défaut
  // code
}
private void readObject(ObjectInputStream s) throws IOException {
  s.defaultReadObject(); // pour le comportement par défaut
  // code
}
```

# Exemple

## ObjetTestSerializablePerso.java

```
public class ObjetTestSerializablePerso implements Serializable {
  private String login;
  private transient String password;
  public ObjetTestSerializablePerso(String login, String password) {
    this.login = login;
    this.password = password;
  }
  private void writeObject(ObjectOutputStream out) throws IOException {
    out.defaultWriteObject();
    out.writeObject(crypte(password));
  }
  private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
    in.defaultReadObject();
    password=crypte((String)in.readObject());
  }
  private String crypte(String chaine) {
    return new StringBuilder(chaine).reverse().toString();
  }
  public String toString() {
    return "login : "+login+" / password : "+password;
  }
}
```

## TestSerializablePerso.java

```
public static void main(String[] args) throws Exception {
  ObjetTestSerializablePerso otsp = new ObjetTestSerializablePerso("nicolas","motdepasse");
  System.out.println(otsp);
  ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("/tmp/fichier"));
  oos.writeObject(otsp);
  oos.close();
  ObjectInputStream ois = new ObjectInputStream(new FileInputStream("/tmp/fichier"));
  otsp = (ObjetTestSerializablePerso)ois.readObject();
  System.out.println(otsp);
  ois.close();
}
```

## Exécution

```
login : nicolas / password : motdepasse
login : nicolas / password : motdepasse
```

## /tmp/fichier

```
$ strings /tmp/fichier
7fr.insarouen.asi.prog.cours6.ObjetTestSerializablePersoL
logint
Ljava/lang/String;xpt
nicolast
essapedtomx
```

# Contrôler totalement la sérialisation : `Externalizable`

## Principe

- la sérialisation standard stocke des informations sur l'objet (les données mais aussi des méta-données : version de la classe, nom et type des attributs, ...) ce qui prend énormément de temps.
- l'implémentation de l'interface `Externalizable` permet de gérer soi-même la sérialisation

## L'interface `Externalizable`

```
public interface Externalizable extends Serializable {
  void writeExternal(ObjectOutput out) throws IOException;
  void readExternal(ObjectInput in) throws IOException, ClassNotFoundException;
}
```

## Attention

Contrairement à la sérialisation classique le constructeur par défaut de l'objet désérialisé est utilisé.

# Exemple

## ObjetTestExternalizable.java

```
public class ObjetTestExternalizable implements Externalizable {
  private int nombre;
  private int nombreAleatoire;
  public ObjetTestExternalizable() {
   nombreAleatoire = (int)(100*Math.random());
   System.err.println("On m'appelle ?");
  }
  public ObjetTestExternalizable(int nombre) {
    this();
    this.nombre = nombre;
  }
  public void writeExternal(ObjectOutput out) throws IOException {
    out.writeInt(nombre);
  }
  public void readExternal(ObjectInput out) throws IOException,ClassNotFoundException {
    nombre = out.readInt();
  }
  public String toString() {
    return "["+super.toString()+"]("+nombreAleatoire+"---"+nombre+")";
  }
}
```

## TestSerialisationExternalizable.java

```
public class TestSerialisationExternalizable {
  public static void main(String[] args) throws Exception {
    ObjetTestExternalizable objTestEx1 = new ObjetTestExternalizable(1);
    ObjetTestExternalizable objTestEx2 = new ObjetTestExternalizable(2);

    System.out.println(objTestEx1);
    System.out.println(objTestEx2);

    ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("/tmp/fichier"));
    oos.writeObject(objTestEx1);
    oos.writeObject(objTestEx1);
    oos.writeObject(objTestEx2);
    oos.close();

    ObjectInputStream ois = new ObjectInputStream(new FileInputStream("/tmp/fichier"));
    System.out.println(ois.readObject());
    System.out.println(ois.readObject());
    System.out.println(ois.readObject());
    ois.close();
  }
}
```

## Exécution

```
On m'appelle ?
On m'appelle ?
[fr.insarouen.asi.prog.cours6.ObjetTestExternalizable@7d772e](47---1)
[fr.insarouen.asi.prog.cours6.ObjetTestExternalizable@11b86e7](99---2)
On m'appelle ?
[fr.insarouen.asi.prog.cours6.ObjetTestExternalizable@19821f](62---1)
[fr.insarouen.asi.prog.cours6.ObjetTestExternalizable@19821f](62---1)
On m'appelle ?
[fr.insarouen.asi.prog.cours6.ObjetTestExternalizable@addbf1](85---2)
```